GLV Management System

Detailed System Documentation and Operator Reference

Document	System documentation
Prepared for	God's Love Ventures / GLV Management System operators
Generated	October 26, 2026
Workspace	C:\Users\andre\glv-management-system
Stack	Next.js 16 App Router, React 19, Auth.js v5, Prisma, Neon Postgres, Tailwind CSS

This document describes the current GLV Management System as implemented in the repository. It intentionally avoids secrets and shows environment variable names only.

1. Executive Overview

GLV Management System is a business operations application for managing layaway customers, product accounts, installment payments, staff, procurement readiness, credits/refunds, reports, settings, notifications, audit logs, and admin support. The system is built as a server-rendered Next.js App Router application with Prisma-backed persistence in Neon Postgres.

Area	What it does
Customer operations	Registers customers, assigns staff, creates product accounts, records payments, tracks balances, and keeps customer/account histories together.
Financial operations	Records payments, generates receipt numbers, recalculates balances, creates overpayment credits, tracks salaries, and exports weekly Excel reports.
Procurement operations	Shows products that should be bought when linked accounts reach at least 70% paid, including fully paid but undelivered accounts. Operators confirm the quantity actually bought and those units leave the list.
Staff operations	Manages staff records, applications, salaries, permissions, login accounts, password reset, and staff-specific dashboards.
Control and governance	Uses role/permission gates, audit logs, admin confirmations, settings, lifecycle rules, and protected server actions.

2. High-Level Architecture

Diagram 1: System Architecture

```
User Browser
   |
   v
Next.js App Router pages and layouts
   |
   +--> Server Components read Prisma data for pages
   +--> Server Actions mutate customers, accounts, payments, products, staff, settings
   +--> Route Handlers expose auth, notifications, weekly export, health, AI support
   |
   v
Auth.js v5 session and route authorization
   |
   v
Prisma Client / Neon Postgres
   |
   +--> AuditLog records important business changes
   +--> Weekly Excel export composes data with ExcelJS
   +--> AI Support route optionally calls Groq when configured
```

Layer	Key files	Responsibility

Presentation shell	app/layout.tsx, components/app-shell.tsx, components/dashboard-nav.tsx	Protected layout, sidebar navigation, role-specific menu visibility, notification badges, admin AI support bubble. The former floating calculator widget and the cloned per-table top scrollbar have both been removed; scrollbars are styled once in globals.css and tabbed pages share components/ui/tabs-nav.tsx.
Pages	app/**/page.tsx	Server-rendered module screens for dashboard, customers, accounts, payments, products, staff, reports, settings, activity, credits, and audit logs.
Mutations	actions/*.ts	Server Actions for authenticated business writes and audit logging.
Domain services	lib/*.ts	Shared business logic for auth config, reports, lifecycle, procurement, account creation, payment recording, settings, permissions, and utilities. lib/prisma.ts exports a lazy proxy so a build needs no database credentials.
Persistence	prisma/schema.prisma, prisma/migrations	Data model and migration history for Neon Postgres.
API routes	app/api/**/route.ts	Auth route, change-password, logout, notifications, weekly export, system health, and AI support.

3. Data Model and Relationships

Diagram 2: Entity Relationship Overview

```
Staff 1 ──< Customer 1 ──< CustomerAccount >── 1 Product
                           |
                           +──< Payment
                           |
                           +──< CustomerCredit

Staff 1 ── 0..1 User
Staff 1 ──< StaffSalaryPayment
Customer 1 ──< CustomerCredit
User / system actions ──< AuditLog
Setting stores singleton-like business configuration
StaffApplication stores pending signup requests before staff/user creation
```

Model	Purpose	Important fields
User	Login identity and role/permission carrier.	email, password, role, staffId, permissions, mustChangePassword, online, lastSeenAt
Staff	Employee profile connected to customers, salaries, and optional user.	code, fullName, email, phone, active, monthlySalary

StaffApplication	Public staff signup request awaiting admin review.	fullName, email, phone, status, reviewedBy, reviewedAt
Customer	Person buying products through layaway.	customerId, fullName, phone, address, nationalId, staffId
Product	Sellable product or combo.	name, category, costPrice, transportCost, layawayPrice, dailyAmount, duration, stockOnHand, active
InventoryMovement	One change to a product's stock on hand.	productId, delta, reason, balanceAfter, note, accountId, createdBy, createdAt
CustomerAccount	A customer's product layaway contract.	targetAmount, dailyAmount, totalPaid, balance, status, deliveryStatus, deliveredAt, deliveredWithBalance, balanceAtDelivery, deliveryNote
Payment	Installment payment record with generated receipt.	receiptNo, accountId, amount, paymentDate, method, receivedBy
CustomerCredit	Overpayment or closure refund credit.	amount, remainingAmount, status, source, accountId, paymentId
StaffSalaryPayment	Payroll payment tracking.	staffId, amount, paymentDate, notes, paidBy
AuditLog	Immutable-style activity history for key changes.	userId, action, entity, entityId, oldValue, newValue
Setting	Company, layaway, payroll, security, notification, theme, and system configuration.	companyName, thresholds, prefixes, payrollDay, permissions-related defaults, status fields

4. Roles, Permissions, and Authentication

Authentication uses Auth.js v5 credentials flow. Route authorization is centralized in the Auth.js authorized callback. Users with mustChangePassword are redirected to /change-password until they complete the first-login reset. Staff are limited to operational routes unless granted explicit permissions.

Role / permission	Access pattern
SUPER_ADMIN	Full administrative role. Treated as admin throughout the UI and server checks.
ADMIN	Administrative access, including settings and privileged modules.
STAFF	Operational access to dashboard, activity, assigned customers/accounts/payments, and change-password.
MANAGE_CUSTOMERS	Allows broader customer management beyond assigned staff ownership.
MANAGE_ACCOUNTS	Allows account creation/correction/delivery operations beyond assigned ownership.

	Delivering before a plan is fully paid additionally requires an admin role.
MANAGE_PAYMENTS	Allows payment management, credits/refunds, and privileged payment workflows.
VIEW_REPORTS	Allows access to reports and weekly export.
MANAGE_PRODUCTS	Allows product catalog access, the procurement module, and the inventory module (receiving stock and correcting counts).
MANAGE_STAFF	Allows staff records, applications, salaries, and password resets.
VIEW_AUDIT_LOGS	Allows read-only audit log access.

Diagram 3: Authentication and Authorization Flow

```
Login
  |
  v
Auth.js credentials provider validates user
  |
  v
JWT callback stores id, role, permissions, staffId, mustChangePassword
  |
  v
Authorized callback checks route
  |
  +--> no session: redirect to /login
  +--> must change password: redirect to /change-password
  +--> staff without permission: redirect to /dashboard
  +--> allowed: render protected route
```

5. Module Catalog

Module	Route(s)	What it does
Public Home	/	Landing/public entry point for GLV.
Login	/login	Credentials login with Auth.js server action integration.
Forgot Password	/forgot-password	Password reset request UI.
Signup	/signup	Staff application submission before admin approval.
Change Password	/change-password, /api/change-password	First-login or forced password change; clears stale auth cookies after success.
Dashboard	/dashboard	Role-shaped summary: admins see animated semicircular KPI gauges where prior-period comparisons are available in a four-column desktop card layout from the large breakpoint, followed by a weekly collections-versus-deposits curved trend and keyed account-status donut; staff see scoped KPIs with

		their own deposit total and trend indicators. Deposit data comes from StaffDeposit by depositDate and does not represent additional customer revenue. Longer-period comparisons live in the trend chart.
Activity	/activity	Operational activity charts show administrator weekly collections as separate curved daily lines for each collecting staff member, with a keyed legend, plus account-status donuts and ranked dot plots. Staff views remain scoped and hide business financial values.
Customers	/customers, /customers/new, /customers/[id], /customers/[id]/edit	Customer CRUD, search/filtering, assignment, profile, account/payment history, and customer-level actions.
Accounts	/accounts, /accounts/new, /accounts/[id]	Product account creation, automatic terms, status, balance, delivery, corrections, lifecycle-gated cancellation/reactivation documents, payment entry, a payment-threshold filter for all eligible pending-delivery accounts, and installment-day progress.
Payments	/payments, /payments/new	Payment recording, receipt numbers, customer receipt communication, grouped history, deletion/recalculation, and staff-safe filtering.
Products	/products, /products/new, /products/[id], /products/[id]/edit	Catalog management, product economics, category badges, and procurement tab.
Procurement	/products?tab=procurement and /products/procurement/[productId]	Buying list for products whose demand exceeds stock on hand. The per-product page is a read-only breakdown marking each account In stock or To buy.
Inventory	/inventory and /inventory/[productId]	Stock on hand per product, receiving stock, correcting a count, and the movement ledger behind every figure.
Staff	/staff, /staff/new, /staff/[id], /staff/[id]/edit	Staff records, permissions, assigned work, salary tracking, deactivation, deletion, and password resets.
Staff Applications	/staff/applications	Admin review queue for signup requests.
Credits & Refunds	/credits	Open credits from overpayments or closures and refund tracking.
Reports	/reports, /api/reports/weekly-export	Admin reports, salary tracking, and Excel workbook export including procurement list.
Audit Logs	/audit-logs	Read-only trail of important system actions.
Settings	/settings?tab=..., /settings/legal, /settings/sms	Super Admin control panel organised as tabs (Company, Business Rules, Payroll, Notifications, Security, Appearance, Product Categories, Data and System). Each tab saves only its own fields.

Notifications	/api/notifications	Attention counts for modules such as procurement, overdue/suspended accounts (overdue computed from the expected end date) and paid-off accounts awaiting delivery; sidebar clears opened notifications locally.
AI Support	/api/support/assistant, lib/ai-support.ts, components/ai-support-chat.tsx	Admin-only Groq-backed support assistant for GLV workflows, with model fallback when a hosted model is retired.
Health/Logout	/api/system/health, /api/logout	Basic health check and logout/cookie cleanup.

6. Customer, Account, Payment, and Delivery Flow

Diagram 4: Customer Account Lifecycle

```
Create customer
  |
  +--> assign staff and generate customer ID
  |
  v
Create product account
  |
  +--> product sets target amount, daily amount, duration, expected end date
  |
  v
Record installment payment
  |
  +--> generate receipt
  +--> increase totalPaid
  +--> reduce balance
  +--> complete account when balance reaches zero
  +--> create CustomerCredit for overpayment
  +--> write AuditLog
  |
  v
Confirm delivery
  |
  +--> fully paid account: any account manager may confirm
  +--> balance still owing: admin only, reason required,
  |      any account status; balanceAtDelivery recorded, account stays open
  |
  +--> deliveryStatus becomes DELIVERED
  +--> stock on hand for the product decreases by one
  |      (admin "past handover": dated, no stock movement)
  +--> account leaves the procurement list
  |
  v
Archive (daily 09:00 lifecycle job plus hourly post-response freshness guard; configured delay
after delivery; 2 days when setting is zero)
  |
  +--> only COMPLETED + DELIVERED + zero balance is archived
```

```
  +--> an ARCHIVED account still owed its product returns to
  |      COMPLETED (awaiting delivery); one owing money returns to ACTIVE
  +--> archived plans still count as completed sales in every report
```

Flow step	Server code	Key controls
Create customer	actions/customers.ts	Duplicate warning, staff assignment, permission/ownership checks, audit log.
Create account	actions/accounts.ts, lib/customer-account-creation.ts	Uses selected product economics, creates expected end date and opening balance.
Automatic terms	lib/customer-documents.ts, lib/customer-communications.ts	Creates addressed terms after account creation and queues one customer channel when contact details exist.
Record payment	actions/payments.ts, lib/payment-recording.ts	Rejects closed/suspended/cancelled/completed accounts, generates receipt, updates balance/status, creates overpayment credit, and queues one receipt channel.
Delete payment	actions/payments.ts	Recalculates account totals and balance; restricted to permitted users.
Correct account price/product	actions/accounts.ts	Admin password confirmation, recalculates balances, can create credit when product/price drops below paid amount.
Confirm delivery	actions/accounts.ts, lib/inventory.ts	Fully paid accounts, COMPLETED or ARCHIVED, are confirmed by any account manager from the account page, the customer profile or the Accounts list (Awaiting delivery filter). Delivering with a balance owing requires an admin role, an explicit opt-in, and a stored reason; it records balanceAtDelivery and leaves the account open. It is available on every account status (canDeliverWithBalance in lib/account-lifecycle-rules.ts): only a positive balance and an undelivered product are required. On a Closed account it voids any OPEN ACCOUNT_CLOSURE_REFUND credit in the same transaction and lists it in the audit entry; on Cancelled, Suspended, Closed or Archived accounts the panel warns that payments are blocked until the status changes. Stock on hand drops by one and a DELIVERED movement is written. An admin can instead record a past handover with its date, which writes no stock movement because the unit already left. Reversing a delivery (Mark pending) returns a unit only when that account's last stock movement took one, and on an archived plan it is admin-only and moves the

		plan back to COMPLETED. If stock is already zero the delivery still records, the count floors at zero, and the operator is warned to correct it. Revalidates account/customer/product/inventory/dashboard screens.
Archive	lib/account-lifecycle.ts, lib/account-lifecycle-rules.ts	A daily 09:00 CRON_SECRET-protected job runs the lifecycle sweep; page and report requests register a post-response freshness guard that keeps active systems fresh at most hourly. A database lease coordinates instances. The setting deliveryTimeAfterCompletionDays controls the delay; zero keeps the two-day default. A completed, delivered, zero-balance account with no delivery date is archived immediately. Misfiled archived accounts return to the correct work list. Writes are conditional on the status read, so overlapping work cannot duplicate a closure credit.
Balance changes	lib/payment-recording.ts, actions/accounts.ts	Payment edit/delete/undo, price override and product correction share getStatusAfterBalanceChange: paying off completes a plan (an archived plan stays archived); owing again reopens a COMPLETED or ARCHIVED plan as ACTIVE so it can be collected.

7. Product, Inventory, and Procurement Module

Inventory and procurement are one loop. Inventory is the stored record of what is physically in the store room: Product.stockOnHand, backed by an InventoryMovement ledger row for every change. Procurement is a computed view over that record, not a separate table. It groups pending-delivery customer accounts by product when their payment progress reaches the effective procurement threshold, currently at least 70% paid including fully paid accounts that are not yet delivered, and then subtracts the units already in stock.

A product therefore appears on the buying list only when the units its customers are owed exceed the units on the shelf; a product whose stock covers its demand does not appear at all. Stock is allocated to the customers closest to finishing their plan first, so the unit on the shelf is reserved for whoever is due it soonest. The allocation rule is the pure function allocateStockToDemand in lib/procurement.ts, covered by scripts/inventory.test.ts.

The Accounts page includes a Payment threshold met (up to 100%) status filter. It selects every pending-delivery account at or above the configured procurement payment threshold, including fully paid accounts. The threshold defaults to 70 percent. Paid Progress displays installment days covered out of the plan duration. This day-based display is separate from the amount threshold used to qualify accounts

for the filter. The threshold compares the actual amount paid with the account target in whole pesewas (cents): at 70 percent, GHS 210 paid toward GHS 1,840 (11.41 percent) is excluded and GHS 1,288.00 (exactly 70 percent) is included. Only pending-delivery accounts that are Active, Overdue, Dormant, Probation or Completed qualify; delivered, closed, archived, cancelled and suspended accounts are excluded. The page re-checks every row against this rule immediately before rendering, so a stale candidate list cannot show an under-threshold account, and the count and pagination reflect only the rows that pass. Stock allocation does not affect the Accounts filter; stock only determines which products and quantities appear on the Products procurement list.

Receiving stock is the action that closes the loop. On the procurement tab or the inventory page the operator presses Receive on a product row and enters the quantity bought; those units enter stock and the buying list reduces by exactly that quantity, so a partial purchase leaves the remainder listed. Delivery moves stock the other way, and a correction sets the count to what was actually on the shelf. Every movement is stored with its reason, the resulting balance, the person and the time, so any figure can be explained afterwards. Receiving and correcting both require MANAGE_PRODUCTS and are audit logged as RECEIVE_STOCK and CORRECT_STOCK. Because stock on hand is part of the shared procurement query, the products tab, the sidebar attention badge, the procurement Excel export, the weekly report sheet and the reports module all reduce together.

This replaced an earlier per-account procured flag and the hand-typed quantity-on-sale field, both dropped by migration 20260917100000_inventory_module. The flag marked units as bought but could not answer how many were actually held, so stock and demand could not be compared. Units marked procured and not yet delivered were converted into opening stock by that migration, attributed to system, so they would not be bought a second time.

Diagram 5: Inventory and Procurement Flow

```
CustomerAccount
  |
  +--> status in ACTIVE / COMPLETED / OVERDUE / DORMANT / PROBATION
  +--> deliveryStatus is PENDING
  +--> totalPaid / targetAmount >= 70%
  |
  v
Units owed, per Product
  |
  +--> subtract Product.stockOnHand, highest paid % served first
  |
  +--> covered by stock --> waiting for handover, NOT on the buying list
  +--> not covered      --> units to buy
  |
  v
Products procurement tab + sidebar notification + weekly report sheet
  |    (a product whose stock covers its demand does not appear at all)
  |
  v
```

```
Receive stock (quantity bought)
  |
  +--> Product.stockOnHand increases
  +--> InventoryMovement RECEIVED written with the new balance
  +--> buying list reduces by exactly that quantity
  |
  v
Confirm delivery on the account completes the handover
  |
  +--> Product.stockOnHand decreases by one
  +--> InventoryMovement DELIVERED written, linked to the account
  +--> zero stock still delivers: floors at zero and warns the operator
```

Procurement field	Meaning
Products to buy	Number of products whose units owed exceed their stock on hand.
Units to buy	Eligible pending-delivery accounts across all products, less the units already in stock.
In stock	Units of that product currently held, shown beside the units to buy so the figure can be read at a glance.
Estimated total cost	Sum of landed product cost for the units still to buy. Units covered by stock are not costed again.
Average paid	Average payment progress across eligible accounts for that product.
Highest paid	Highest payment progress across eligible accounts, used for sorting.
Receive	Operator enters the quantity actually bought in a dialog on the product row. Those units enter stock and leave the buying list immediately.
Clearing rule	A product leaves the buying list when stock on hand covers the units its customers are owed, or when delivery confirmation changes deliveryStatus to DELIVERED.
Movement reasons	OPENING, RECEIVED, DELIVERED, RETURNED, and CORRECTION. Each row stores the signed change, the resulting balance, an optional note, the linked account where relevant, the person and the time.

8. Staff, Applications, Salary, and Password Management

Staff can enter through an application workflow or be created directly by an admin. Approved staff can receive a user account, staff code, permissions, and a temporary/reset password. Salary tracking records monthly salary obligations and salary payments for reports.

Diagram 6: Staff Application and Management Flow

```
Public signup
  |
  v
```

```
StaffApplication PENDING
  |
  +--> Admin approves: create Staff + optional User + audit log
  |
  +--> Admin rejects: mark REJECTED + audit log
  |
  v
Staff profile
  |
  +--> assigned customers/accounts/payments
  +--> monthly salary and salary payments
  +--> password reset and permission edits
```

Capability	Details
Staff list/detail	Shows staff code, contact info, active status, assigned customers, and operational work.
Applications	Admin queue for public signup requests with approve/reject outcomes.
Permissions	Granular permissions can expand staff access to admin sections without changing role to admin.
Salary	Monthly salary stored on Staff; salary payment records support report balances and payroll summaries.
Password reset	Admin reset produces temporary credentials and can require password change.
Deletion/deactivation	Destructive staff operations are admin-gated and protected by confirmation logic where applicable.

9. Credits, Refunds, Lifecycle, and Closure Rules

Credits are created when a payment overpays an account or when lifecycle closure creates a refund credit after deducting the service fee. Lifecycle refresh checks active-like accounts and moves them toward DORMANT, PROBATION, or CLOSED based on inactivity.

Rule	Current behavior
Dormant	Account becomes DORMANT after 21 days without payment activity.
Probation	Account becomes PROBATION after 4 months without payment activity.
Closed	Account becomes CLOSED after 6 months without payment activity.
Activity clock	Inactivity is measured from the latest of start date, last payment date, and last reactivation date (CustomerAccount.reactivatedAt).
Paid-off precision	When a payment settles the remaining amount, the balance is set to exactly zero and the account becomes COMPLETED. Payment recording and lifecycle refresh treat floating-point residue up to 0.000000001 as zero; a backdated payment still uses its payment date for lifecycle timing.

Closure refund	When closing with paid amount and no prior closure credit, the configured refund deduction is applied (32% when the saved setting is zero) and the remainder becomes an open credit.
Reactivation	An admin may reactivate a DORMANT, PROBATION or CLOSED account once 6 months of inactivity have passed. The configured refund deduction (32% when zero) is charged against the paid amount, open closure-refund credits are voided, and reactivatedAt is stamped so the ladder restarts from that date.
Overpayment credit	Payment amount beyond remaining balance creates CustomerCredit linked to the payment/account.
Refund action	Credits can be marked refunded by permitted payment/admin users.
Retraction window	Account reactivation, payment deletion, staff deposit deletion, salary deletion, and credit refunds are retractable by an administrator for 3 hours. Each writes a ReversibleAction row inside the action's own transaction; the reverse handler refuses when the record changed after the action rather than overwriting newer work. Retraction writes an UNDO_<ACTION> audit entry.

10. Reports, Weekly Export, and Activity

The reports page is organised as an anchor-navigated sequence of sections: Overview, Analytics, Staff Performance, Deposits, Salaries, and Products. Overview presents its figures in four labelled groups - Collections and Banking, Receivables and Exposure, Payroll, and Profitability - rather than one undifferentiated grid, and colours each figure by whether it is favourable, adverse, or worth watching. The wide ledger tables pin the staff or product column while the money columns scroll sideways.

Reports combine customer/account/payment/staff/salary data into admin-facing summaries and an Excel workbook. A daily 09:00 CRON_SECRET-protected lifecycle job refreshes statuses, with post-response freshness guards during active use; the weekly export registers its guard before building sheets for executive summary, staff performance, accounts, weekly payment history, product profitability, procurement list, and weekly ledger.

Diagram 7: Reporting and Export Flow

```
Report page
  |
  +--> getAdminReportSummary()
  +--> getWeeklyStaffPerformanceReport()
  |
  v
Weekly export route
  |
  +--> register post-response lifecycle freshness guard
  +--> query staff, customers, accounts, payments, salaries, products
  +--> getProcurementList()
  |
```

```
  v
Excel workbook
  |
  +--> Executive Summary
  +--> Staff Performance
  +--> Customer Accounts
  +--> Payment History
  +--> Product Profitability
  +--> Procurement List
  +--> Weekly Ledger
```

Report component	Purpose
Admin report summary	High-level business metrics, balances, status counts, salaries, collection data.
Staff performance report	Ranks staff by collection and assigned account/customer activity.
Activity report	Dashboard/activity charts and recent payment/customer/account signals.
Weekly Excel export	Downloadable workbook intended for operations review and management reporting.
Procurement sheet	Products ready to buy with quantity, costs, layaway price, average paid %, and highest paid %.

11. Notifications, Customer Communications, and AI Support

The system has computed in-app attention items for operators and persisted outbound customer messages. Customer documents and receipts queue exactly one enabled channel. Legal/document messages prefer email, then WhatsApp, then SMS; payment receipts prefer WhatsApp, then SMS, then email. With no email or phone, no outbound row is queued and staff use the in-system receipt/tracking record. Delivery preserves the selected channel and retries provider failures.

Diagram 8: Notification Clear-on-Open Flow

```
AppShell loads protected page
  |
  v
GET /api/notifications
  |
  +--> if user can manage products, compute procurement list
  |
  v
Sidebar badge on Products
  |
  +--> click opens /products?tab=procurement
  +--> store local dismissal signature
  +--> hide same badge until count/label changes
```

Feature	Implementation

Notification source	app/api/notifications/route.ts
Badge rendering	components/dashboard-nav.tsx
Dismissal storage	components/app-shell.tsx localStorage key glv-dismissed-attention
AI support	Admin-only floating chat bubble; route calls Groq chat completions only when GROQ_API_KEY is configured.
AI support scope	Navigation, permissions, payments, accounts, products, procurement, staff, reports, settings, and troubleshooting.
Customer message queue	CustomerMessage records are created by lib/customer-communications.ts and dispatched by the notifications route or daily cron.
No-contact handling	No message is queued; staff explain terms verbally and show the receipt/product tracking information.

12. Settings Module

Settings is the broad admin control panel. It persists company profile, layaway defaults, fees, procurement threshold, payroll defaults, receipt/customer/staff code prefixes, security toggles, notification preferences, theme settings, and system status labels. Effective rules are listed below; administration fee and commission percentages remain saved-only until their calculation bases are defined.

Settings group	Examples	Current use
Company profile	companyName, tradingName, tagline, phone, address, tax IDs	Used for branding and records; visible in shell and settings.
Layaway rules	installmentDurationDays, defaultDailyCollection, minimumDeposit, deliveryTimeAfterCompletionDays	Minimum deposit is enforced on a plan's first payment; delivery setting controls the archive delay (two-day fallback when zero).
Refunds	refundDeductionPercent	Applied to closure refunds and dormant-account reactivation charges; zero retains the 32% fallback.
Procurement	procurementThresholdPercent	Default and effective procurement readiness floor is 70%.
Payroll	defaultMonthlySalary, commission fields, payrollDay	Salary defaults and reporting support. Commission settings are saved-only until the calculation base and payroll treatment are defined.
IDs and receipts	receiptPrefix, customerIdPrefix, staffCodeLength	Receipt generation and ID/code helper logic.

Security	passwordLength, sessionTimeoutMinutes, requirePasswordChange, twoFactorEnabled	Session timeout is checked against the current saved setting on authenticated requests; password-change and 2FA enforcement remain active.
Legal documents	editable terms/cancellation/reactivation/receipt templates	Terms are generated for new accounts and managed in Settings; lifecycle calculations are account-stage gated.
Notifications	email/sms/WhatsApp toggles	Controls the live one-channel customer queue. Provider credentials are required for delivery.
Appearance/status	theme, colors, loadingAnimation, currentVersion, databaseStatus	Stored and partly consumed in UI branding/status surfaces.

13. Auditability and Data Safety

The system records audit logs for major business writes. Important destructive or financial operations are gated by roles, permissions, staff ownership checks, admin password confirmation, confirmation text, or transaction boundaries. Staff-facing data is shaped server-side so restricted financial totals are not merely hidden in the UI.

Control	Where it appears
Server-side auth	auth config, page/session checks, server actions
Permissions	UserPermission enum and hasPermission checks
Ownership restrictions	Staff can act on assigned customers/accounts unless broader permission exists
Transactions	Payments, account corrections, staff/customer/product destructive flows, lifecycle closure
Audit logs	Create/update/delete/payment/lifecycle/settings/staff actions
Credential safety	Secrets are environment variables; documentation must never print values.
Duplicate-submission prevention	Every form submit control disables itself and shows a pending state while its server action runs (components/ui/submit-button.tsx for shadcn Button, components/ui/plain-submit-button.tsx for bespoke button markup, both backed by useFormStatus), so a slow request can no longer be triggered twice by repeated clicks. Payment, customer, and account creation additionally carry a client-generated Idempotency-Key so a resubmitted request cannot create a second record server-side.

14. Deployment, Verification, and Operations

Command	Purpose	Current note

npm run lint	ESLint project check.	Passed in latest verification.
npx tsc --noEmit	TypeScript check without emitting files.	Passed in latest verification.
npx next build	Next production build without migration deploy.	Passed in latest verification.
npm run build	Runs prisma generate and next build.	Does not deploy migrations.
npm run db:deploy	Deploy migrations explicitly before a dependent release.	Use DATABASE_URL_UNPOOLED from a trusted operator environment; do not use the port 6543 transaction pooler.
npm run seed	Run Prisma seed script.	Use only when intentionally bootstrapping data.

Environment variable names used by the system include DATABASE_URL, AUTH_SECRET, AUTH_URL, GROQ_API_KEY, GROQ_MODEL, MNOTIFY_API_KEY, MNOTIFY_SENDER_ID, and CRON_SECRET. Values are intentionally omitted from this document.

15. Important Caveats and Known Follow-Ups

Item	Current status / recommendation
Screenshots	Live authenticated screenshots were not embedded because the in-app browser connector failed in this Windows session before opening. Diagrams are included instead.
Settings effectiveness	Many settings persist today; verify downstream consumers before claiming a setting changes behavior.
Notification persistence	Notifications clear locally by signature; there is no database notification/read model yet.
AI support persistence	Chat state is browser-local and not audited or stored.
Prisma package config	Prisma warns that package.json#prisma is deprecated for Prisma 7; not currently blocking.
Database reachability	Full build and migration deploy require Supabase connectivity through the correct pooler.
Unstaged local file	app/page.tsx had an unrelated local edit at the time this documentation was generated; it is not part of the pushed procurement/reporting release.

16. Automatic SMS Notifications

BMS Africa provides SMS using the mNotify Quick Bulk SMS API. Configure MNOTIFY_API_KEY, MNOTIFY_SENDER_ID (approved GLV sender GODS LOVE V), and CRON_SECRET server-side. Apply the SmsNotification migration before enabling SMS Notifications in Settings. Never expose the API key or log request URLs containing it.

Super administrators configure six database-backed message templates on Settings > SMS: Salary payment, Customer welcome, 70% progress, Missed payment, Weekly summary - target met, and Weekly summary - below target. The page lists the placeholders permitted for each event, renders a sample preview, validates a 612-character maximum, and can restore all defaults. Saved templates apply to newly queued messages; existing queue rows keep their original message snapshot. Salary resolves to the staff member on the salary payment. Customer templates resolve to the qualifying customer. A customer or staff member with no usable phone number is skipped entirely: no message is queued, none is attempted, and nothing is sent to a fallback recipient.

The weekly summary has two wordings and the system picks one per customer. The expected weekly amount is the daily amount of every plan that customer is still collecting on, multiplied by seven. Paying at or above that amount sends the target-met wording. Paying less sends the below-target wording, which carries the expected amount and the shortfall and refers the customer to their assigned staff member. The praise wording is therefore never sent to a customer who fell short. The comparison is performed in pesewas so floating-point noise cannot turn an exact week into a shortfall, and a customer with no collecting plan counts as on target.

Default message text is branded Rock Frost Group, while every staff/customer name placeholder renders only the first whitespace-delimited name. The BMS handset sender is Rock Frost, which fits the provider's 11-character limit and must be approved in the BMS account before production configuration.

A saved salary payment queues one staff SMS. A new product account queues one welcome SMS at its start date. Payment totals at or above 70 percent queue one milestone SMS per account. Payment edits also check the milestone. No historical salary or welcome messages are backfilled. These alerts always use SMS, independently of the existing document and receipt channel preference.

Recording a staff deposit triggers the weekly customer summary. For that staff member, GLV totals each customer's Payment records from Monday through Sunday across all product accounts and queues one summary only for customers whose total is greater than zero. The dedupe key combines staff, customer and week. Another deposit cannot duplicate an accepted message, but it can refresh a summary that is still Pending or Failed. Customers never receive another staff member's totals.

Active and overdue accounts with a positive balance qualify for reminders after fourteen full days (two weeks) without payment, and at most once per further two-week period thereafter, so the sequence is day 14, day 28, day 42. The later of account start, latest payment date and latest payment entry time anchors the interval. Backdated entries restart it. Other lifecycle statuses are excluded. The Missed payment template supports staffName, resolved to the first name of the staff member assigned to that customer. The protected /api/cron/sms-notifications job runs at 09:00 UTC daily, queues reminders and dispatches up to 100 messages in groups of five. Authenticated activity and qualifying mutations also dispatch pending messages. Monitor backlog and schedule more frequent runs if volume requires it.

The dedicated /settings/sms page lets super administrators enable or pause automatic SMS, verify provider/API/sender configuration, review the five notification rules, inspect the latest 100 queue records, and retry definitive failures. The queue records unique event keys within the business transaction. Conditional claims prevent concurrent workers sending the same item. Before dispatch, source existence, account eligibility and current phone are checked. Correct missing phones or provider errors before Retry. Provider acceptance is labelled ACCEPTED, not confirmed handset delivery. Check BMS campaign history using the saved campaign ID. Rate limits retry after one hour, up to five attempts. Timeouts and interrupted workers become UNKNOWN and require reconciliation rather than blind resending. Restored pending or processing messages become UNKNOWN. Disabling SMS pauses the queue.

Implementation checks use mocked provider/database tests in scripts/sms.test.ts, TypeScript, lint and a build without migration deployment. Production migration, credentials, authenticated UI and real delivery must be verified separately before activation is considered complete.

17. Integrated Business Management

The administrator-only /business workspace presents People, Payroll, Accounting and Analytics as four connected sections of one GLV workflow. It uses the existing Staff, Payment and StaffSalaryPayment records and adds leave requests, performance reviews and operating expenses. In Accounting, administrators can review the full expense history or filter by date range, and edit a cashbook entry to correct a mistake; each edit stores before and after values and the editor in the audit log. No module activation or duplicate employee setup is required.

Section	Shared records and behavior
People	Existing staff directory plus leave requests, approve or reject decisions, and dated performance reviews scored 1 to 5.
Payroll	Existing monthly salary and salary payment history. Recording payment retains validation, audit logging and the staff-only salary SMS.
Accounting	Customer payments are cash inflow. Salary payments and BusinessExpense records are cash outflow. Staff deposits reconcile collections and are not counted twice. Administrators can review expenses across all dates or a selected date range and edit entries; edits are audit logged with before and after values.
Analytics	Current month revenue, payroll paid and outstanding, expenses, net cash, headcount, pending leave, review average and all-time cash position. A six-month curved cash-movement chart compares customer collections, payroll and expenses; a keyed donut shows current-month expense categories. Net cash is not accounting profit.

Apply migration 20260909183000_integrated_business_management before deploying the page. The version 1 application backup and restore workflow does not yet contain leave, performance review or

business expense tables. Retain database-level backups and do not rely on an older application backup to recover these records until a versioned restore upgrade is released.

Appendix A. Source File Map

Area	Representative files
Pages	app/dashboard/page.tsx; app/customers/page.tsx; app/accounts/page.tsx; app/payments/page.tsx; app/products/page.tsx; app/staff/page.tsx; app/reports/page.tsx; app/settings/page.tsx
Server actions	actions/customers.ts; actions/accounts.ts; actions/payments.ts; actions/products.ts; actions/staff.ts; actions/settings.ts; actions/salaries.ts; actions/applications.ts
Domain libraries	lib/auth.config.ts; lib/reports.ts; lib/weekly-excel-report.ts; lib/procurement.ts; lib/payment-recording.ts; lib/customer-account-creation.ts; lib/account-lifecycle.ts
UI components	components/app-shell.tsx; components/dashboard-nav.tsx; components/customer-form.tsx; components/account-form.tsx; components/payment-form.tsx; components/product-form.tsx; components/dashboard-analytics.tsx; components/activity-analytics-charts.tsx; components/reports/chart-primitives.tsx (shared SVG curved trend, comparison line, donut, dot plot, tooltip, and legend charts); components/reports/meter-gauge.tsx; components/reports/analytics-charts.tsx
API routes	app/api/notifications/route.ts; app/api/reports/weekly-export/route.ts; app/api/support/assistant/route.ts; app/api/change-password/route.ts; app/api/system/health/route.ts
Data model	prisma/schema.prisma; prisma/migrations/*